Team notebook

UAlgException

October 15, 2026

Contents

1 Básicos **1**
- 1.1 2DPrefixSums . . . 1
- 1.2 BinarySearchOnAnswer . . . 1
- 1.3 CoordinateCompression . . . 2
- 1.4 CustomHash . . . 2
- 1.5 OrderedSet . . . 2

2 DP **3**
- 2.1 ArrayDescription . . . 3
- 2.2 BookShop . . . 3
- 2.3 CoinsCombinations2 . . . 4
- 2.4 CountingTowers . . . 4
- 2.5 EditDistance . . . 4
- 2.6 LongestCommomSubsequence . . . 5
- 2.7 MinimalGridPath . . . 5
- 2.8 MinimizingCoins . . . 6
- 2.9 RemovalGames . . . 6
- 2.10 template . . . 7

3 GameTheory **8**
- 3.1 MEX . . . 8
- 3.2 MisereNim . . . 8
- 3.3 Nim . . . 8
- 3.4 StaircaseNim . . . 9

4 Geometry **9**
- 4.1 Counter-Clockwise . . . 9
- 4.2 Point . . . 9
- 4.3 PolygonArea . . . 10
- 4.4 SegmentIntersection . . . 10

5 Grafos **10**
- 5.1 BFS . . . 10
- 5.2 Bellman$_{F}ord$ . . . 10
- 5.3 BinaryLifting . . . 11
- 5.4 DFS . . . 11
- 5.5 DFS$_{c}iclos$ . . . 11
- 5.6 Dijkstra . . . 12
- 5.7 Disjointset . . . 12
- 5.8 Edmonds-Karp . . . 13
- 5.9 Floyd-Warshall . . . 14
- 5.10 Hierholzer . . . 15
- 5.11 Korasaju . . . 15
- 5.12 Kruskal . . . 16
- 5.13 TWO$_{S}AT$ . . . 17
- 5.14 khan . . . 18

6 Math **18**
- 6.1 BinomialCoefficients . . . 18
- 6.2 ExtendedGCD . . . 19
- 6.3 GCDandLCM . . . 19
- 6.4 GaussianElimination . . . 20
- 6.5 MathUtils . . . 20
- 6.6 MatrixExponentiation . . . 21
- 6.7 ModularExponentiation . . . 21
- 6.8 ModularInverse . . . 21
- 6.9 SieveAndFactorization . . . 22

7 RangeQuery **22**
7.1 BinaryIndexedTree 22
7.2 LazySegTree 23
7.3 RangeMinimum 24

8 SlidingWindow **25**
8.1 Generic 25
8.2 Minimum 26

9 Strings **26**
9.1 PrefixFunction 26
9.2 PrefixTree 27
9.3 RollingHashing 27
9.4 Z-Algorithm 28

1 Básicos

1.1 2DPrefixSums

```
/*
    ================================================================================
 * Algoritmo: 2D Prefix Sums
 *  Descrio :  Pr -computa a soma de qualquer submatriz em O(N*M).
 * A consulta de uma rea (r1, c1) at (r2, c2)   feita em O(1).
 *
    ================================================================================
    */
void build_2d_prefix(const vector<vector<int>>& grid, vector<vector<long
    long>>& pref) {
    int R = grid.size(), C = grid[0].size();
    // pref[i][j] guarda a soma de (0,0) at (i-1, j-1)
    for (int i = 1; i <= R; i++) {
        for (int j = 1; j <= C; j++) {
            pref[i][j] = grid[i-1][j-1]
                       + pref[i-1][j]
                       + pref[i][j-1]
                       - pref[i-1][j-1];
        }
    }
}

// Retorna a soma do retngulo com canto superior esquerdo (r1, c1) e
    inferior direito (r2, c2)
// Coordenadas 0-indexed
long long query_2d(const vector<vector<long long>>& pref, int r1, int c1,
    int r2, int c2) {
    return pref[r2+1][c2+1] - pref[r1][c2+1] - pref[r2+1][c1] +
    pref[r1][c1];
}
```

1.2 BinarySearchOnAnswer

```
/*
    ================================================================================
 * Algoritmo: Binary Search on Answer
 *  Descrio : Encontra o primeiro valor no intervalo [low, high] que
    satisfaz
 * a  funo  check(). Estrutura imune a off-by-one errors.
 *
    ================================================================================
 */
bool check(long long mid) {
    // Implementar a lgica de  validao  do problema para o valor 'mid'
    return true;
}

long long binary_search_answer(long long low, long long high) {
    long long ans = -1;
    while (low <= high) {
        long long mid = low + (high - low) / 2;  // Previne overflow
        if (check(mid)) {
            ans = mid;       // Guarda o candidato vlido
            high = mid - 1;  // Tenta encontrar um valor ainda menor
    (ajustar
                             // conforme o problema)
        } else {
            low = mid + 1;  // O valor atual  invlido, procura mais acima
        }
    }
    return ans;
}
```

1.3 CoordinateCompression

```
/*
    ================================================================================
 * Algoritmo: Coordinate Compression
 *  Descrio : Mapeia valores esparsos [10, 10^9, 50, ...] para [0, 2, 1,
    ...].
 *
    ================================================================================
 */
void coordinate_compression(vector<long long>& a) {
    vector<long long> sorted_a = a;
    sort(sorted_a.begin(), sorted_a.end());
    // Remove duplicados
    sorted_a.erase(unique(sorted_a.begin(), sorted_a.end()),
    sorted_a.end());

    // Substitui cada elemento pelo seu rank (ndice comprimido)
    for (int i = 0; i < a.size(); i++) {
        a[i] = lower_bound(sorted_a.begin(), sorted_a.end(), a[i]) -
               sorted_a.begin();
    }
}
```

1.4 CustomHash

```
/*
    ================================================================================
 * Algoritmo: Custom Hash para Unordered Map
 *  Descrio : Previne testes anti-hash que causam TLE em competies.
 * Uso: unordered_map<long long, int, custom_hash> safe_map;
 *
    ================================================================================
 */
#include <chrono>
struct custom_hash {
    static uint64_t splitmix64(uint64_t x) {
        x += 0x9e3779b97f4a7c15;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
        x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
        return x ^ (x >> 31);
    }

    size_t operator()(uint64_t x) const {
        static const uint64_t FIXED_RANDOM =
            chrono::steady_clock::now().time_since_epoch().count();
        return splitmix64(x + FIXED_RANDOM);
    }
};
```

1.5 OrderedSet

```
/*
    ================================================================================
 * Algoritmo: Ordered Set (PBDS)
 *  Descrio : Um std::set com superpoderes.
 * Funcionalidades extra em O(log N):
 *  - find_by_order(k): retorna um iterador para o k-simo menor elemento
    (0-indexed)
 *  - order_of_key(k): retorna o nmero de elementos estritamente menores
    que k
 *
    ================================================================================
    */
#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>

using namespace std;
using namespace __gnu_pbds;

//  Definio  do Ordered Set
typedef tree<int, null_type, less<int>, rb_tree_tag,
    tree_order_statistics_node_update> ordered_set;

// NOTA: Para funcionar como um multiset (permitir duplicados), muda
    `less<int>` para `less_equal<int>`.
// Cuidado que no less_equal, a funo erase() normal apaga TODAS as
    instncias.
// Deves apagar por iterador: s.erase(s.find_by_order(s.order_of_key(x)));
```

2 DP

2.1 ArrayDescription

```
/*
    ================================================================================
 * Problema: Array Description
 * Algoritmo: DP Local (Adjacencia)
 * Descricao: Modificar um array preenchendo zeros, sendo que valores
 * adjacentes podem diferir no maximo em 1. Conta todas as combinacoes
    validas.
 *
    ================================================================================
 */
int solve(int arraySize, int upperBound, int array[]) {
    // memo[i][k] = n de formas de criar prefixo i terminado no valor k
    vector<vi> memo(arraySize + 1, vi(upperBound + 1, 0));

    // Caso base: primeiro elemento do array (i=1)
    for (int k = 1; k <= upperBound; k++) {
        if (array[0] == k || array[0] == 0) {
            memo[1][k] = 1;
        }
    }

    // Transicao para os restantes elementos
    for (int i = 2; i <= arraySize; i++) {
        for (int k = 1; k <= upperBound; k++) {
            // Se a posicao nao e zero, apenas transita para o valor fixo
            if (array[i - 1] != 0 && array[i - 1] != k) {
                memo[i][k] = 0;
                continue;
            }
            // Soma as 3 opcoes do vizinho anterior: [k-1, k, k+1]
            for (int prev = k - 1; prev <= k + 1; prev++) {
                if (prev < 1 || prev > upperBound) continue;
                memo[i][k] = (memo[i][k] + memo[i - 1][prev]) % MOD;
            }
        }
    }

    int ans = 0;
    for (int k = 1; k <= upperBound; k++) {
        ans = (ans + memo[arraySize][k]) % MOD;
    }
    return ans;
}
```

2.2 BookShop

```
/*
    ================================================================================
 * Problema: Book Shop
 * Algoritmo: Mochila 0/1 (0/1 Knapsack)
 *  Descrio : Dado um array de preos e outro de pginas, encontrar o mximo
 * de pginas  possvel  comprar sem exceder um preo mximo.
 *
    ================================================================================
 */
int solve(vi pages, vi prices, int maxTotalPrice) {
    int n = pages.size();
    // memo[i][sz] = mx pginas usando os primeiros i livros com oramento
    sz
    vector<vi> memo(n + 1, vi(maxTotalPrice + 1, 0));

    for (int i = 1; i <= n; i++) {
        int pi = prices[i - 1], pa = pages[i - 1];

        for (int sz = 1; sz <= maxTotalPrice; sz++) {
            //  Opo  1:  No  comprar o livro atual
            memo[i][sz] = memo[i - 1][sz];

            //  Opo  2: Comprar o livro atual (se o oramento permitir)
            if (sz >= pi && memo[i - 1][sz - pi] + pa > memo[i][sz]) {
                memo[i][sz] = memo[i - 1][sz - pi] + pa;
            }
        }
    }
    return memo[n][maxTotalPrice];
}
```

2.3 CoinsCombinations2

```
/*
    ================================================================================
 * Problema: Coin Combinations II
 * Algoritmo: Unbounded Knapsack (Contagem Combinatria)
 *  Descrio :  Nmero  de formas de atingir um alvo. A ORDEM DAS MOEDAS NO
 * IMPORTA (ex: 2+1  igual a 1+2). Para isso, iteramos as moedas no ciclo
 * externo!
```

```
 *
    ================================================================================
 */
int solve(int target, vi coins) {
    vi memo(target + 1, 0);
    memo[0] = 1;  // 1 forma de fazer o valor 0 (conjunto vazio)

    // O facto de a  iterao  das moedas ser externa  o que previne
     permutaes
    for (int c : coins) {
        for (int i = 0; i <= target; i++) {
            if (c <= i) {
                memo[i] = (memo[i] + memo[i - c]) % MOD;
            }
        }
    }
    return memo[target];
}
```

2.4 CountingTowers

```
/*
    ================================================================================
 * Problema: Counting Towers
 * Algoritmo: State Machine DP
 *  Descrio :  Nmero  de formas de construir uma torre 2 x N.
 * Estados: 0 = blocos superiores separados; 1 = bloco superior nico
    (unido).
 *
    ================================================================================
 */
ll memo[1000000 + 1][2];

//  Pr -computa  at  10^6 para responder a mltiplas queries rapidamente
void precompute() {
    memo[1][0] = 1;  // Linha 1 dividida
    memo[1][1] = 1;  // Linha 1 unida

    for (int i = 2; i <= 1e6; i++) {
        // Formas de prolongar/cortar blocos separados e unidos
        memo[i][0] = (4 * memo[i - 1][0] + memo[i - 1][1]) % MOD;
        memo[i][1] = (2 * memo[i - 1][1] + memo[i - 1][0]) % MOD;
    }
}

int solve() {
    int t;
    cin >> t;
    precompute();  // Executar apenas uma vez!

    while (t--) {
        int n;
        if (!(cin >> n)) return 1;
        cout << (memo[n][0] + memo[n][1]) % MOD << '\n';
    }
    return 0;
}
```

2.5 EditDistance

```
/*
    ================================================================================
 * Problema: Edit Distance (Distncia de Levenshtein)
 * Algoritmo: DP 2D Baseado em Strings
 *  Descrio :  Nmero  mnimo  de  inseres ,  remoes  e  substituies  para
 * transformar a string base na target.
 *
    ================================================================================
 */
int solve() {
    string base, target;
    getline(cin, base);
    getline(cin, target);

    if (base == target) return 0;
    if (target.empty()) return base.length();
    if (base.empty()) return target.length();

    int n = base.length(), m = target.length();
    vector<vi> memo(n + 1, vi(m + 1, 0));

    // Casos base: transformar de/para strings vazias
    for (int i = 1; i <= n; i++) memo[i][0] = i;
    for (int i = 1; i <= m; i++) memo[0][i] = i;

    for (int i = 1; i <= n; i++) {
```

```
        for (int j = 1; j <= m; j++) {
            if (base[i - 1] == target[j - 1]) {
                memo[i][j] = memo[i - 1][j - 1];  // Sem custo
            } else {
                memo[i][j] = min({
                                 memo[i - 1][j - 1],  // Substituio
                                 memo[i - 1][j],      // Remoo
                                 memo[i][j - 1]       // Insero
                             }) +
                             1;
            }
        }
    }
    return memo[n][m];
}
```

2.6 LongestCommomSubsequence

```
/*
    ================================================================================
 * Problema: Longest Common Subsequence
 * Algoritmo: DP 2D Classico + Backtracking
 * Descricao: Encontra a maior subsequencia comum entre dois vetores e
    imprime
 * o comprimento e os elementos dessa subsequencia, reconstituindo o
    caminho.
 *
    ================================================================================
 */
void solve(vi base, vi target) {
    int n = base.size(), m = target.size();
    vector<vi> memo(n + 1, vi(m + 1, 0));

    // 1. Construir a Tabela DP
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (base[i - 1] == target[j - 1]) {
                memo[i][j] = memo[i - 1][j - 1] + 1;
            } else {
                memo[i][j] = max(memo[i][j - 1], memo[i - 1][j]);
            }
        }
    }

    // 2. Reconstruir a Subsecuencia (Backtracking do fim para o inicio)
    vi result;
    int i = n, j = m;
    while (i > 0 && j > 0) {
        if (base[i - 1] == target[j - 1]) {
            result.push_back(base[i - 1]);
            i--;
            j--;
        }
        // Desloca para a direcao de onde veio o valor maximo
        else if (memo[i - 1][j] > memo[i][j - 1]) {
            i--;  // O valor otimo veio de cima
        } else {
            j--;  // O valor otimo veio da esquerda
        }
    }

    cout << result.size() << '\n';
    // A resposta e recolhida de tras para a frente
    for (int k = result.size() - 1; k >= 0; k--) {
        cout << result[k] << (k == 0 ? "\n" : " ");
    }
}
```

2.7 MinimalGridPath

```
/*
    ================================================================================
 * Problema: Minimal Grid Path
 * Algoritmo: Breadth-First Search (BFS) guiada por nvel
 *  Descrio : Encontra o caminho lexicograficamente menor (formando uma
    string)
 * numa grelha N x N. Usa BFS sncrona para manter apenas as melhores opes
    por
 *  nvel .
 *
    ================================================================================
 */
string solve(vector<string> grid) {
    int n = grid.size();
    vector<vector<bool>> visited(n, vector<bool>(n, false));
    vector<pair<int, int>> active;
```

```
    string result = "";
    result += grid[0][0];
    active.push_back({0, 0});
    visited[0][0] = true;

    // O comprimento do caminho  exatamente 2*N - 2 passos
    for (int step = 0; step < 2 * n - 2; ++step) {
        char min_char = 'Z' + 1;  // Valor infinito (acima de 'Z')
        vector<pair<int, int>> next_candidates;

        // Avalia vizinhos de todas as clulas timas do nvel atual
        for (auto [r, c] : active) {
            if (r + 1 < n) {
                if (grid[r + 1][c] < min_char) min_char = grid[r + 1][c];
                next_candidates.push_back({r + 1, c});
            }
            if (c + 1 < n) {
                if (grid[r][c + 1] < min_char) min_char = grid[r][c + 1];
                next_candidates.push_back({r, c + 1});
            }
        }

        result += min_char;
        active.clear();

        // Mantm para o prximo passo apenas os vizinhos que tm o min_char
        for (auto [r, c] : next_candidates) {
            if (grid[r][c] == min_char && !visited[r][c]) {
                visited[r][c] = true;
                active.push_back({r, c});
            }
        }
    }
    return result;
}
```

2.8 MinimizingCoins

```
/*
    ================================================================================
 * Problema: Minimizing Coins
 * Algoritmo: Unbounded Knapsack (Minimizao)
 *  Descrio : Encontrar o nmero  mnimo  de moedas para atingir um valor
    alvo.
 * Moedas podem ser usadas infinitas vezes.
 *
    ================================================================================
 */
int solve(int target, vi coins) {
    // Inicializar com INF, pois queremos o mnimo
    vi memo(target + 1, INF);
    memo[0] = 0;  // 0 moedas para fazer o valor 0

    for (int i = 1; i <= target; i++) {
        for (int c : coins) {
            if (c <= i) {
                // Transio :  Mnimo  entre  no  usar e usar a moeda 'c'
                memo[i] = min(memo[i], memo[i - c] + 1);
            }
        }
    }

    // Se o valor continua INF,  impossvel atingir o alvo
    return (memo[target] < INF ? memo[target] : -1);
}
```

2.9 RemovalGames

```
/*
    ================================================================================
 * Problema: Removal Game
 * Algoritmo: Range DP (DP em Intervalos)
 *  Descrio : Dois jogadores apanham itens extremos do array
    alternadamente.
 * Maximizar a  pontuao  do primeiro jogador.
 * Percorre os intervalos (gaps) por ordem crescente de comprimento.
 *
    ================================================================================
 */
ll solve(int listSize, vi list) {
    int n = listSize;
    // scores[i][j] armazena o melhor delta (Score_1 - Score_2) para
    [i...j]
    vector<vector<ll>> scores(n, vector<ll>(n, 0));
```

```
    // g   o "gap" ou tamanho do intervalo (j - i)
    for (int g = 0; g < n; ++g) {
        for (int i = 0, j = g; j < n; ++i, ++j) {
            if (g == 0) {
                // Intervalo de tamanho 1
                scores[i][j] = list[i];
            } else if (g == 1) {
                // Intervalo de tamanho 2
                scores[i][j] = max(list[i], list[j]);
            } else {
                //  Opo  1: apanhar list[i]. O adversrio  forar  o pior
    caso
                // para ns
                ll val1 = list[i] + min(scores[i + 2][j], scores[i + 1][j
    - 1]);

                //  Opo  2: apanhar list[j]. O adversrio  forar  o pior
    caso
                // para ns
                ll val2 = list[j] + min(scores[i + 1][j - 1], scores[i][j
    - 2]);

                scores[i][j] = max(val1, val2);
            }
        }
    }
    return scores[0][n - 1];
}
```

2.10 template

```
/*
    ================================================================================
 * Template: Estrutura Base para Competies
 *  Descrio :  Otimizao  de I/O, macros essenciais e constantes comuns.
 *
    ================================================================================
 */
#include <bits/stdc++.h>
using namespace std;

// Atalhos de tipos para poupar teclado (essencial em DP)
using ll = long long;
using vi = vector<int>;
using vll = vector<long long>;
using pii = pair<int, int>;

// Macros teis para manter o cdigo limpo
#define pb push_back
#define all(x) begin(x), end(x)
#define sz(x) (int)(x).size()

// Constantes comuns
const int INF = 1e9;
const ll LINF = 1e18;
const int MOD = 1e9 + 7;

void solve() {
    // Lgica  do problema aqui
    int n;
    if (!(cin >> n)) return;
}

int main() {
    //  Otimizao  crucial de Input/Output (Evita Time Limit Exceeded)
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int t = 1;
    // cin >> t; // Descomentar se houver mltiplos test cases

    while (t--) {
        solve();
    }
    return 0;
}
```

3 GameTheory

3.1 MEX

```
/*
    ================================================================================
 * Algoritmo: Teorema de Sprague-Grundy (Funcao MEX)
 * Descricao: Calcula o valor de Grundy para um jogo imparcial qualquer.
```

```
 * MEX (Minimum Excluded) e o menor inteiro nao negativo ausente num
    conjunto.
 *
    ============================================================================
 */
// Funcao para encontrar o MEX de um conjunto de valores (estados
    seguintes)
int mex(const vector<int>& next_states_grundy_values) {
    unordered_set<int> s(next_states_grundy_values.begin(),
                         next_states_grundy_values.end());
    for (int i = 0;; i++) {
        if (!s.count(i)) return i;
    }
}

// Exemplo de calculo de Grundy com Memoizacao (DP)
// 'moves' e o conjunto de jogadas validas (ex: retirar 1, 3 ou 4 pedras)
int grundy(int n, const vector<int>& moves, vector<int>& memo) {
    if (n == 0) return 0;  // Posicao perdedora
    if (memo[n] != -1) return memo[n];

    vector<int> next_states;
    for (int m : moves) {
        if (n >= m) {
            next_states.push_back(grundy(n - m, moves, memo));
        }
    }

    return memo[n] = mex(next_states);
}
// Para resolver o jogo, calculas o Grundy de cada pilha individualmente e
// fazes o XOR-Sum de todos os resultados. Se for != 0, o 1o jogador
    ganha.
```

3.2 MisereNim

```
/*
    ============================================================================
 * Variante: Misere Nim
 * Descricao: Jogo do Nim normal, mas o ULTIMO jogador a fazer um
    movimento
 * PERDE.
 *
    ============================================================================
 */
bool misere_nim_winner(const vector<int>& piles) {
    int xor_sum = 0;
    bool all_ones = true;

    for (int x : piles) {
        xor_sum ^= x;
        if (x > 1) all_ones = false;
    }

    // Se todas as pilhas tem apenas 1 pedra, ganha quem deixa um numero
    impar
    // de pilhas
    if (all_ones) {
        return piles.size() % 2 == 0;
    }

    // Caso contrario, a estrategia e exatamente igual ao Nim normal
    return xor_sum != 0;
}
```

3.3 Nim

```
/*
    ============================================================================
 * Algoritmo: Teoria de Jogos (Nim e Sprague-Grundy)
 *  Descrio :  Avaliao  de estados vencedores em jogos imparciais.
 *
    ============================================================================
 */
#include <bits/stdc++.h>
using namespace std;

// Jogo do Nim Clssico: N pilhas de pedras. Os jogadores alternam
    retirando
// 1 ou mais pedras de uma nica pilha. O ltimo a jogar ganha.
// Regra de Ouro: O 1 jogador ganha se e s se o XOR-Sum for diferente de
    0.
bool nim_game_winner(const vector<int>& piles) {
    int xor_sum = 0;
    for (int x : piles) {
```

```
        xor_sum ^= x;
    }
    return xor_sum !=
           0;  // true se o jogador que vai iniciar tem estratgia
    vencedora
}
```

3.4 StaircaseNim

```
/*
    ================================================================================
 * Variante: Staircase Nim (Nim em Escadas)
 * Descricao: Os jogadores movem moedas do degrau 'i' para 'i-1'. As
    moedas
 * no degrau 0 sao removidas do jogo.
 * Regra: E equivalente a um Nim classico onde APENAS os degraus IMPARES
 * contam para o XOR-Sum.
 *
    ================================================================================
 */
bool staircase_nim_winner(const vector<int>& stairs) {
    int xor_sum = 0;
    // Assume-se que stairs[1] e o primeiro degrau, stairs[2] o segundo,
    etc.
    for (int i = 1; i < stairs.size(); i += 2) {
        xor_sum ^= stairs[i];
    }
    return xor_sum != 0;
}
```

4 Geometry

4.1 Counter-Clockwise

```
/*
    ================================================================================
 * Teste de  Orientao
 * Retorna:
 *  1 se C est    esquerda de AB (Curva  esquerda)
 * -1 se C est    direita de AB (Curva  direita)
 *  0 se A, B e C so colineares
 *
    ================================================================================
 */
int orientation(Point a, Point b, Point c) {
    long long val = (b - a).cross(c - a);
    if (val > 0) return 1;
    if (val < 0) return -1;
    return 0;
}
```

4.2 Point

```
/*
    ================================================================================
 * Base de Geometria: Struct Point
 *  Operaes  usam 'long long' para evitar overflow em multiplicaes.
 *
    ================================================================================
 */
struct Point {
    long long x, y;

    //  Subtrao  de pontos cria um vetor (A -> B = B - A)
    Point operator-(const Point& p) const { return {x - p.x, y - p.y}; }

    // Produto Vetorial (Cross Product): (this X p)
    long long cross(const Point& p) const { return x * p.y - y * p.x; }
};
```

4.3 PolygonArea

```
/*
    ================================================================================
 * rea de Polgono (Shoelace Formula)
 * Os pontos devem estar ordenados (horrio ou anti-horrio).
 * Retorna o dobro da rea (para manter sempre em inteiros).
 * Se precisarem da rea real, dividam por 2.0 no fim.
 *
    ================================================================================
 */
```

```
long long polygon_area_x2(const vector<Point>& poly) {
    long long area = 0;
    int n = poly.size();
    for (int i = 0; i < n; i++) {
        // poly[i] cruza com o vrtice seguinte (fazendo wrap-around para
    0)
        area += poly[i].cross(poly[(i + 1) % n]);
    }
    return abs(area);  // A rea real  abs(area) / 2.0
}
```

4.4 SegmentIntersection

```
/*
    ================================================================================
 *  Interseco  de Segmentos (Simples)
 * Verifica se os segmentos AB e CD se cruzam (exclui casos de
    colinearidade).
 *
    ================================================================================
 */
bool segments_intersect(Point a, Point b, Point c, Point d) {
    int o1 = orientation(a, b, c);
    int o2 = orientation(a, b, d);
    int o3 = orientation(c, d, a);
    int o4 = orientation(c, d, b);

    // Se as  orientaes  forem opostas, h  interseco
    return (o1 != o2) && (o3 != o4);
}
```

5 Grafos

5.1 BFS

```
#include <bits/stdc++.h>
using namespace std;

vector<int> BFS(vector<vector<int>> &list)
{
    int size = list.size();
    vector<bool> visited(size, false);
    vector<int> path;
    queue<int> q;

    for (int i = 0; i < size; i++)
    {
        if (!visited[i])
        {
            visited[i] = true;
            q.push(i);
            while (!q.empty())
            {
                int current = q.front();
                path.push_back(current);
                q.pop();
                for (int adj : list[current])
                {
                    if (!visited[adj])
                    {
                        visited[adj] = true;
                        q.push(adj);
                    }
                }
            }
        }
    }
    return path;
}
```

5.2 Bellman$_{F}ord$

```
#include <bits/stdc++.h>
using namespace std;

vector<long long> dist;

bool Bellman_Ford(const vector<aresta>& list, int n) {
    bool flag = false;
    for (int i = 0; i < n; i++) {
        for (const aresta& atual : list) {
            int s = atual.s;
            int f = atual.f;
```

```
            long long w = atual.w;

            if (dist[s] != LLONG_MAX && dist[f] > dist[s] + w) {
                dist[f] = dist[s] + w;
                if (i == n - 1) {
                    flag = true;
                    break;
                }
            }
        }
    }
    return flag;
}
```

5.3 BinaryLifting

```
#include <bits/stdc++.h>
using namespace std;


int n;
const int nMax = 30;

vector<vector<int>> build_up(const vector<int>& jumps)
{
    vector<vector<int>> up(nMax, vector<int>(n, 0));

    for (int i = 0; i < n; i++)
    {
        up[0][i] = jumps[i];
    }

    for (int i = 1; i < nMax; i++)
    {
        for (int j = 0; j < n; j++)
        {
            up[i][j] = up[i - 1][up[i - 1][j]];
        }
    }

    return up;
}
```

```
int BinaryLifting(int current, const vector<vector<int>>& up, int f)
{
    for (int i = 0; i < nMax; i++)
    {
        if ((f & (1 << i)) != 0)
        {
            current = up[i][current];
        }
    }
    return current;
}
```

5.4 DFS

```
#include <bits/stdc++.h>
using namespace std;

vector<bool> visited;
vector<int> path;
void DFS(int current, vector<vector<int>> list) {
    visited[current] = true;
    path.push_back(current);
    for(int adj: list[current]) {
        if(!visited[adj]) {
            DFS(adj,list);
        }
    }
}
```

5.5 DFS$_{ciclos}$

```
#include <bits/stdc++.h>
using namespace std;

vector<bool> na_chamada;
int inicio=-1;
int fim =-1;
vector<bool> visited;
vector<int> origem;

void dfs(int current, vector<vector<int>>&list) {
```

```
    visited[current] = true;
    na_chamada[current] = true;
    for(int adj: list[current]) {
        if(!visited[adj]) {
            origem[adj] = current;
            dfs(adj, list);
            if(inicio!=-1) break;
        } else if(na_chamada[adj]) {
            inicio= adj;
            fim = current;
            return;
        }
    }
    na_chamada[current] = false;
}
```

5.6 Dijkstra

```
#include <bits/stdc++.h>
using namespace std;
struct aresta
{
    int v;
    long long w;
};
struct CompareAresta
{
    bool operator()(aresta &a, aresta &b)
    {
        return a.w > b.w;
    }
};
vector<long long> dist;
void Dijkstra(vector<vector<aresta>> list, int start)
{
    dist.assign(list.size(), LLONG_MAX);
    priority_queue<aresta, vector<aresta>, CompareAresta> pq;
    dist[start] = 0;
    pq.push({start, 0});
    while (!pq.empty())
    {
        aresta atual = pq.top();
        pq.pop();
        int v = atual.v;
        long long w = atual.w;
        if (w > dist[v])
            continue;
        for (aresta adj : list[v])
        {
            int vadj = adj.v;
            long long wadj = adj.w;
            if (dist[vadj] > dist[v] + wadj)
            {
                dist[vadj] = dist[v] + wadj;
                pq.push({vadj, dist[vadj]});
            }
        }
    }
}
```

5.7 Disjointset

```
#include <bits/stdc++.h>
using namespace std;

class DSU
{
private:
    vector<int> parent, rank, tamanho;

public:
    int nMax, nComponents;

    DSU(int size)
    {
        parent.resize(size + 1);
        rank.assign(size + 1, 0);
        tamanho.assign(size + 1, 1);
        for (int i = 0; i <= size; i++)
        {
            parent[i] = i;
        }
        nMax = 0;
        nComponents = size;
    }
    int find(int i)
```

```
    {
        if (parent[i] == i)
            return i;
        return parent[i] = find(parent[i]);
    }

    void uniao(int i, int j)
    {
        int iRoot = find(i);
        int jRoot = find(j);
        if (jRoot == iRoot)
            return;

        if (rank[iRoot] < rank[jRoot])
        {
            parent[iRoot] = jRoot;
            tamanho[jRoot] += tamanho[iRoot];
            nMax = max(nMax, tamanho[jRoot]);
        }
        else if (rank[iRoot] > rank[jRoot])
        {
            parent[jRoot] = iRoot;
            tamanho[iRoot] += tamanho[jRoot];
            nMax = max(nMax, tamanho[iRoot]);
        }
        else
        {
            parent[iRoot] = jRoot;
            tamanho[jRoot] += tamanho[iRoot];
            nMax = max(nMax, tamanho[jRoot]);
            rank[jRoot]++;
        }
        nComponents--;
    }

public:
    int getnComponents()
    {
        return nComponents;
    }

public:
    int getnMax()
    {
        return nMax;
    }
};
```

5.8 Edmonds-Karp

```
#include <bits/stdc++.h>
using namespace std;

class DistinctRoutesSolver
{
public:
    int n;
    vector<vector<int>> capacity;
    vector<vector<int>> graphResidual; // Para fluxo BFS
    vector<vector<int>> graphOriginal; // Para reconstruir o caminho
    int maxFlowValue;

    DistinctRoutesSolver(int size)
    {
        n = size;
        capacity.assign(n, vector<int>(n, 0));
        graphResidual.resize(n);
        graphOriginal.resize(n);
        maxFlowValue = 0;
    }

    void addEdge(int u, int v) {
        graphResidual[u].push_back(v);
        graphResidual[v].push_back(u);
        graphOriginal[u].push_back(v);
        capacity[u][v]++;
    }

    int computeMaxFlow(int s, int t) {
        maxFlowValue = 0;
        while (true) {
            vector<int> parent(n, -1);
            queue<int> q;
            q.push(s);
            parent[s] = -2;

            while (!q.empty()) {
                int curr = q.front();
```

```
            q.pop();
            if (curr == t) break;

            for (int next : graphResidual[curr]) {
                if (parent[next] == -1 && capacity[curr][next] > 0) {
                    parent[next] = curr;
                    q.push(next);
                }
            }
        }

        if (parent[t] == -1) break; // No h mais caminhos
        maxFlowValue++;
        int curr = t;

        while (curr != s) {
            int prev = parent[curr];
            capacity[prev][curr]--;
            capacity[curr][prev]++;
            curr = prev;
        }
    }
    return maxFlowValue;
}

vector<vector<int>> getPaths(int s, int t) {
    vector<vector<int>> paths;
    for (int i = 0; i < maxFlowValue; i++) {
        vector<int> path;
        int curr = s;
        path.push_back(s);
        while (curr != t) {
            bool moved = false;
            for (int next : graphOriginal[curr]) {
                if (capacity[curr][next] == 0 && capacity[next][curr]
>= 1) {
                    path.push_back(next);
                    capacity[next][curr]--;
                    capacity[curr][next]++;
                    curr = next;
                    moved = true;
                    break;
                }
            }
            if (!moved) break;
            }
            paths.push_back(path);
        }
        return paths;
    }
};
```

5.9 Floyd-Warshall

```
#include <bits/stdc++.h>
using namespace std;
vector<vector<long long>> graph;
//graph[i][i] =0
void Warshall(int n)
{
    for (int k = 0; k < n; k++)
    {
        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < n; j++)
            {
                if (graph[i][k] != LLONG_MAX && graph[k][j] != LLONG_MAX)
                {
                    graph[i][j] = min(graph[i][j], graph[i][k] +
    graph[k][j]);
                }
            }
        }
    }

}
```

5.10 Hierholzer

```
#include <bits/stdc++.h>
using namespace std;

vector<vector<pair<int, int>>> graph;
int n, m;
vector<int> degree;
vector<int> circuit;
```

```
// Para grafos direcionados deve usar in_degree e out_degree.
// Os valores no indice i devem ser iguais.
//
void Heirholzer()
{
    cin >> n >> m;
    graph.resize(n + 1);
    for (int i = 0; i < m; i++)
    {
        int a, b;
        cin >> a >> b;
        graph[a].push_back({b, i});
        graph[b].push_back({a, i});
        degree[a]++;
        degree[b]++;
    }
    for (int i = 1; i <= n; i++)
    {
        if (degree[i] % 2 != 0)
        {
            cout << "IMPOSSIBLE";
            return;
        }
    }
    vector<int> currPath;
    currPath.push_back(1);
    vector<bool> visited(m, false);

    while (currPath.size() > 0)
    {
        int currNode = currPath[currPath.size() - 1];
        bool edge_found = false;
        while (!graph[currNode].empty())
        {
            auto [nextNode, edge_idx] = graph[currNode].back();
            graph[currNode].pop_back();

            if (!visited[edge_idx])
            {
                visited[edge_idx] = true;
                currPath.push_back(nextNode);
                edge_found = true;
                break;
            }
        }
        if (!edge_found)
        {
            circuit.push_back(currNode);
            currPath.pop_back();
        }
    }
    if (circuit.size() != m + 1)
    {
        cout << "IMPOSSIBLE";
        return;
    }
    reverse(circuit.begin(), circuit.end());
}
```

5.11 Korasaju

```
#include <bits/stdc++.h>
using namespace std;

vector<vector<int>> dag;
vector<int> in_degree;
vector<int> out_degree;

vector<vector<int>> graph, graphR;
vector<bool> visited;
vector<int> scc_id;
int scc_count;
vector<int> order;
vector<vector<int>> sccs;

void dfs(int start, vector<vector<int>> &list)
{
    visited[start] = true;
    for (int v : list[start])
    {
        if (!visited[v])
            dfs(v, list);
    }
    order.push_back(start);
}

void dfs1(int start, vector<vector<int>> &list)
```

```
{
    visited[start] = true;
    scc_id[start] = scc_count;
    sccs.back().push_back(start);
    for (int v : list[start])
    {
        if (!visited[v])
        {
            dfs1(v, list);
        }
    }
}

void kosaraju(int n)
{
    sccs.clear();
    scc_count = 0;
    visited.assign(n + 1, false);
    for (int i = 1; i <= n; i++)
    {
        if (!visited[i])
            dfs(i, graph);
    }
    visited.assign(n + 1, false);
    scc_id.assign(n + 1, 0);
    vector<int> conexos;
    for (int i = order.size() - 1; i >= 0; i--)
    {
        int u = order[i];
        if (!visited[u])
        {
            sccs.push_back({});
            dfs1(u, graphR);
            conexos.push_back(u);
            scc_count++;
        }
    }
}

void build_condensation_graph(int n)
{
    dag.assign(scc_count, vector<int>());
    in_degree.assign(scc_count, 0);
    out_degree.assign(scc_count, 0);
    set<pair<int, int>> edges;

    for (int u = 1; u <= n; u++)
    {
        for (int v : graph[u])
        {
            int id_u = scc_id[u];
            int id_v = scc_id[v];

            if (id_u != id_v && !edges.count({id_u, id_v}))
            {
                edges.insert({id_u, id_v});
                dag[id_u].push_back(id_v);
                out_degree[id_u]++;
                in_degree[id_v]++;
            }
        }
    }
}
```

5.12 Kruskal

```
struct aresta
{
    int s, f;
    long w;
};

long long kruskal(int n, vector<aresta>& edges)
{
    long long cost = 0, count = 0;

    sort(edges.begin(), edges.end(),
         [](const aresta& a, const aresta& b)
         {
             return a.w < b.w;
         });

    DSU dsu(n);

    for (const aresta& edge : edges)
    {
```

```
        int s = edge.s;
        int f = edge.f;
        long long w = edge.w;

        if (dsu.find(s) != dsu.find(f))
        {
            dsu.unionSet(s, f);
            cost += w;

            if (++count == n - 1)
                return cost;
        }
    }

    return -1;
}
```

5.13 TWO$_S$AT

```
#include <bits/stdc++.h>
using namespace std;

int n, m;
vector<vector<int>> graph, graphR;
vector<bool> visited;
vector<int> scc_id;
vector<int> order;
int scc_count;

void dfs(int start)
{
    visited[start] = true;
    for (int v : graph[start])
    {
        if (!visited[v])
            dfs(v);
    }
    order.push_back(start);
}

void dfs1(int start)
{
    visited[start] = true;
    scc_id[start] = scc_count;
    for (int v : graphR[start])
    {
        if (!visited[v])
            dfs1(v);
    }
}

int node(char sign, int val)
{
    return sign == '+' ? val : val + m;
}
int neg(int val)
{
    return val > m ? val - m : val + m;
}

void criarGrafos()
{
    for (int i = 0; i < n; i++)
    {
        char sign1, sign2;
        int val1, val2;
        cin >> sign1 >> val1 >> sign2 >> val2;

        int u = node(sign1, val1);
        int v = node(sign2, val2);

        int negu=neg(u);
        int negv=neg(v);

        graph[negu].push_back(v);
        graphR[v].push_back(negu);

        graph[negv].push_back(u);
        graphR[u].push_back(negv);
    }
}

bool twosat()
{
    scc_count = 0;
    for (int i = 1; i <= 2 * m; i++)
    {
        if (!visited[i])
```

```
            dfs(i);
    }
    visited.assign(2 * m + 1, false);

    for (int i = 2 * m - 1; i >= 0; i--)
    {
        int u = order[i];
        if (!visited[u])
        {
            scc_count++;
            dfs1(u);
        }
    }
    string result = "";
    for(int i = 1; i<=m;i++){
        if(scc_id[i] == scc_id[i+m]){
            return false;
        }
        if(scc_id[i] > scc_id[i+m]){
            result += "+ ";
        }else {
            result +="- ";
        }
    }
    return true;
}
```

5.14 khan

```
#include <bits/stdc++.h>
using namespace std;

vector<int> order;
vector<vector<int>> graph;


void topoSortKhan()
{
    int n = graph.size()-1;
    vector<int> indegree(n+1,0);
    queue<int> q;
    for (int i = 1; i <= n; i++)
    {
        for (int next : graph[i])
        {
            indegree[next]++;
        }
    }

    for (int i = 1; i <= n; i++)
    {
        if (indegree[i] == 0)
        {
            q.push(i);
        }
    }

    while (!q.empty())
    {
        int top = q.front();
        q.pop();
        order.push_back(top);
        for (int next : graph[top])
        {
            indegree[next]--;
            if (indegree[next] == 0)
                q.push(next);
        }
    }
}
```

6 Math

6.1 BinomialCoefficients

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
const int MOD = 1'000'000'007;
const int MAXN = 1'000'000 + 5;

ll fact[MAXN], invFact[MAXN];

/*
```

```
 * Pr - computao de fatoriais para responder combinaes em O(1).
 */
void precompute_factorials() {
    fact[0] = 1;
    invFact[0] = 1;
    for (int i = 1; i < MAXN; i++) {
        fact[i] = (fact[i - 1] * i) % MOD;
    }

    invFact[MAXN - 1] = 1;
    ll last = fact[MAXN - 1];
    ll invLast = 1;
    // Usando o inverso modular do ltimo fatorial
    ll power = MOD - 2;
    ll base = last % MOD;
    while (power > 0) {
        if (power & 1) invLast = (invLast * base) % MOD;
        base = (base * base) % MOD;
        power >>= 1;
    }
    invFact[MAXN - 1] = invLast;

    for (int i = MAXN - 2; i >= 1; i--) {
        invFact[i] = (invFact[i + 1] * (i + 1)) % MOD;
    }
}

ll nCr(int n, int r) {
    if (r < 0 || r > n) return 0;
    return fact[n] * invFact[r] % MOD * invFact[n - r] % MOD;
}
```

6.2 ExtendedGCD

```
/*
    ================================================================================
 * Algoritmo: Algoritmo de Euclides Estendido (Extended GCD)
 *  Descrio : Encontra x e y tais que A*x + B*y = gcd(A, B).
 * Retorna o Maior Divisor Comum (GCD) de A e B.
 *
    ================================================================================
 */
#include <bits/stdc++.h>
using namespace std;

long long extGCD(long long a, long long b, long long& x, long long& y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    long long x1, y1;
    long long d = extGCD(b, a % b, x1, y1);
    x = y1;
    y = x1 - y1 * (a / b);
    return d;
}

// Inverso modular universal (funciona mesmo se M no for primo, desde que
// gcd(A, M) == 1)
long long universalModInverse(long long a, long long m) {
    long long x, y;
    long long g = extGCD(a, m, x, y);
    if (g != 1) {
        return -1;  // Inverso no existe se A e M no forem coprimos
    }
    return (x % m + m) % m;
}
```

6.3 GCDandLCM

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

/*
 * Mximo divisor comum e mnimo mltiplo comum.
 */
ll gcd(ll a, ll b) {
    return b == 0 ? a : gcd(b, a % b);
}

ll lcm(ll a, ll b) {
    return (a / gcd(a, b)) * b;
}
```

6.4 GaussianElimination

```cpp
/*
    ================================================================================
 * Algoritmo: Eliminacao de Gauss
 * Descricao: Resolve sistemas de equacoes lineares.
 * Retorna: 1 (Solucao Unica), 0 (Sem Solucao), 2 (Infinitas Solucoes).
 * O array 'ans' e preenchido com os valores das variaveis.
 *
    ================================================================================
 */
#include <bits/stdc++.h>
using namespace std;

const double EPS = 1e-9;

int gauss(vector<vector<double>> a, vector<double>& ans) {
    int n = (int)a.size();
    int m = (int)a[0].size() - 1;
    vector<int> where(m, -1);

    for (int col = 0, row = 0; col < m && row < n; ++col) {
        int sel = row;
        for (int i = row; i < n; ++i) {
            if (abs(a[i][col]) > abs(a[sel][col])) sel = i;
        }
        if (abs(a[sel][col]) < EPS) continue;

        for (int i = col; i <= m; ++i) swap(a[sel][i], a[row][i]);
        where[col] = row;

        for (int i = 0; i < n; ++i) {
            if (i != row) {
                double c = a[i][col] / a[row][col];
                for (int j = col; j <= m; ++j) a[i][j] -= a[row][j] * c;
            }
        }
        ++row;
    }

    ans.assign(m, 0);
    for (int i = 0; i < m; ++i) {
        if (where[i] != -1) ans[i] = a[where[i]][m] / a[where[i]][i];
    }
    for (int i = 0; i < n; ++i) {
        double sum = 0;
        for (int j = 0; j < m; ++j) sum += ans[j] * a[i][j];
        if (abs(sum - a[i][m]) > EPS) return 0;  // 0 solucoes
    }

    for (int i = 0; i < m; ++i) {
        if (where[i] == -1) return 2;  // Infinitas solucoes
    }
    return 1;  // 1 solucao
}
```

6.5 MathUtils

```cpp
/*
    ================================================================================
 * Arquivos separados para os algoritmos matemticos do MathUtils.cpp
 *
 * - ModularExponentiation.cpp
 * - ModularInverse.cpp
 * - BinomialCoefficients.cpp
 * - SieveAndFactorization.cpp
 * - GCDandLCM.cpp
 * - MatrixExponentiation.cpp
 *
 * Cada algoritmo foi movido para o seu prprio arquivo para facilitar
    reutilizao,
 * leitura e  manuteno .
 *
    ================================================================================
 */
```

6.6 MatrixExponentiation

```cpp
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
const ll MOD = 1'000'000'007LL;
using Matrix = vector<vector<ll>>;
```

```
/*
 *  Multiplicao  de matrizes em O(n^3).
 */
Matrix multiply(const Matrix& A, const Matrix& B) {
    int rA = (int)A.size();
    int cA = (int)A[0].size();
    int cB = (int)B[0].size();

    Matrix C(rA, vector<ll>(cB, 0));
    for (int i = 0; i < rA; i++) {
        for (int k = 0; k < cA; k++) {
            for (int j = 0; j < cB; j++) {
                C[i][j] = (C[i][j] + A[i][k] * B[k][j]) % MOD;
            }
        }
    }
    return C;
}

/*
 *  Exponenciao  de matrizes - O(M^3 log P)
 */
Matrix matrixPower(Matrix A, ll p) {
    int n = (int)A.size();
    Matrix res(n, vector<ll>(n, 0));
    for (int i = 0; i < n; i++) res[i][i] = 1;

    while (p > 0) {
        if (p & 1) res = multiply(res, A);
        A = multiply(A, A);
        p >>= 1;
    }
    return res;
}
```

6.7 ModularExponentiation

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
const ll MOD = 1'000'000'007LL;

/*
 *  Exponenciao  modular rpida - O(log b)
 * Calcula (a^b) % m eficientemente.
 */
ll binpow(ll a, ll b, ll m = MOD) {
    a %= m;
    ll res = 1;
    while (b > 0) {
        if (b & 1) res = (res * a) % m;
        a = (a * a) % m;
        b >>= 1;
    }
    return res;
}
```

6.8 ModularInverse

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
const ll MOD = 1'000'000'007LL;

/*
 * Inverso multiplicativo modular - O(log m)
 * Requisito: m deve ser primo.
 */
ll modInverse(ll n, ll m = MOD) {
    if (n % m == 0) return 0; // evita diviso por zero em casos invlidos
    return binpow(n, m - 2, m);
}

ll binpow(ll a, ll b, ll m = MOD) {
    a %= m;
    ll res = 1;
    while (b > 0) {
        if (b & 1) res = (res * a) % m;
        a = (a * a) % m;
        b >>= 1;
    }
    return res;
}
```

6.9 SieveAndFactorization

```cpp
#include <bits/stdc++.h>
using namespace std;

const int MAX_SIEVE = 1'000'000 + 5;
int spf[MAX_SIEVE];

/*
 * Crivo de Eratstenes para calcular o menor fator primo de cada nmero.
 */
void sieve() {
    for (int i = 1; i < MAX_SIEVE; i++) spf[i] = i;
    for (int i = 2; i * i < MAX_SIEVE; i++) {
        if (spf[i] == i) {
            for (int j = i * i; j < MAX_SIEVE; j += i) {
                if (spf[j] == j) spf[j] = i;
            }
        }
    }
}

vector<int> getFactorization(int x) {
    vector<int> ret;
    while (x != 1) {
        ret.push_back(spf[x]);
        x /= spf[x];
    }
    return ret;
}
```

7 RangeQuery

7.1 BinaryIndexedTree

```cpp
/*
    ================================================================================
 * Algoritmo: Segment Tree com Lazy Propagation (Soma em Intervalo)
 * Complexidade: O(log N) para updates e queries. Memria O(4*N).
 *  Descrio : Resolve  atualizaes  e consultas em ranges (ex: [L, R]).
 * Para mudar para Mximo/Mnimo: alterar as  funes  de merge no
    update/query.
 *
    ================================================================================
 */
struct LazySegTree {
    int n;
    vector<long long> tree, lazy;

    LazySegTree(int n) {
        this->n = n;
        tree.assign(4 * n, 0);
        lazy.assign(4 * n, 0);
    }

    // Empurra a  atualizao  pendente para os ns filhos
    void push(int node, int left, int right) {
        if (lazy[node] != 0) {
            tree[node] += lazy[node] * (right - left + 1);
            if (left != right) {
                lazy[node * 2] += lazy[node];
                lazy[node * 2 + 1] += lazy[node];
            }
            lazy[node] = 0;
        }
    }

    void update(int node, int left, int right, int ql, int qr, long long
    val) {
        push(node, left, right);
        if (left > right || left > qr || right < ql) return;

        if (left >= ql && right <= qr) {
            lazy[node] += val;
            push(node, left, right);
            return;
        }

        int mid = left + (right - left) / 2;
        update(node * 2, left, mid, ql, qr, val);
        update(node * 2 + 1, mid + 1, right, ql, qr, val);

        tree[node] = tree[node * 2] + tree[node * 2 + 1];
    }

    long long query(int node, int left, int right, int ql, int qr) {
        push(node, left, right);
```

```
        if (left > right || left > qr || right < ql) return 0;

        if (left >= ql && right <= qr) return tree[node];

        int mid = left + (right - left) / 2;
        long long res_left = query(node * 2, left, mid, ql, qr);
        long long res_right = query(node * 2 + 1, mid + 1, right, ql, qr);

        return res_left + res_right;
    }

    //  Funes  de interface limpas para chamar na main() (0-indexed)
    void update(int l, int r, long long val) { update(1, 0, n - 1, l, r,
    val); }
    long long query(int l, int r) { return query(1, 0, n - 1, l, r); }
};
```

7.2 LazySegTree

```
/**
 * Segment Tree Recursiva com Lazy Propagation.
 * Ideal para: Range Updates (atualizar um intervalo inteiro) e Range
    Queries.
 * Complexidade: O(log N) para update e query.
 * Espaco: 4 * N.
 */
public static class LazySegTree {
    // 'tree': Armazena o valor agregado (soma, min, max) do intervalo.
    // 'lazy': Armazena o valor de atualizacao pendente que AINDA NAO foi
    repassado para os filhos.
    long[] tree, lazy;
    int n;

    public LazySegTree(int n) {
        this.n = n;
        // O tamanho 4*N e uma cota superior segura para uma arvore
    binaria completa
        // que cobre N elementos.
        tree = new long[4 * n];
        lazy = new long[4 * n];
    }

    /**
 * A funcao PUSH (Propagate).
 * Esta e a magica da Lazy Propagation. Antes de descer para os
filhos de um no,
 * verificamos se ha alguma atualizacao pendente ("preguicosa") neste
no.
 * Se houver, nos a aplicamos aos filhos AGORA e limpamos a preguica
do pai.
 */
private void push(int node, int start, int end) {
    // Se nao houver nada pendente (0), nao faz nada.
    if (lazy[node] != 0) {

        // Se nao for um no folha (tem filhos), propaga a informacao.
        if (start != end) {
            int mid = (start + end) / 2;

            // --- Filho Esquerdo (2*node) ---
            // 1. Repassa a responsabilidade (o valor lazy) para o
filho
            lazy[2 * node] += lazy[node];
            // 2. Atualiza o valor REAL (tree) do filho
imediatamente, para que
            //    futuras consultas nele estejam corretas sem
precisar descer mais.
            //    NOTA: Multiplicamos pelo tamanho do intervalo ((mid
- start + 1)) porque e SOMA.
            //          Se fosse MINIMO ou MAXIMO, nao multiplicaria,
apenas somaria.
            tree[2 * node] += lazy[node] * (mid - start + 1);

            // --- Filho Direito (2*node + 1) ---
            lazy[2 * node + 1] += lazy[node];
            tree[2 * node + 1] += lazy[node] * (end - mid);
        }

        // Como ja empurramos a atualizacao para baixo, limpamos a
"preguica" do no atual.
        lazy[node] = 0;
    }
}

// Funcao publica para chamar a atualizacao (interface amigavel)
// Adiciona 'val' a todos os elementos no intervalo [l, r]
public void update(int l, int r, int val) {
    update(1, 0, n - 1, l, r, val);
```

```
}

// Funcao interna recursiva de atualizacao
// node: indice atual na arvore | start, end: intervalo que este no
cobre
// l, r: intervalo que queremos atualizar | val: valor a adicionar
private void update(int node, int start, int end, int l, int r, int
val) {
    // Caso 1: O intervalo do no atual esta totalmente FORA do
intervalo de atualizacao.
    if (l > end || r < start) return;

    // Caso 2: O intervalo do no atual esta TOTALMENTE DENTRO do
intervalo de atualizacao.
    // Aqui acontece a otimizacao: Atualizamos este no e marcamos o
'lazy' para nao descer mais.
    if (l <= start && end <= r) {
        lazy[node] += val; // Acumula a preguica
        tree[node] += (long) val * (end - start + 1); // Atualiza o
valor deste no
        return; // RETORNA IMEDIATAMENTE! Nao desce para os filhos
(O(log N)).
    }

    // Caso 3: Intersecao parcial. Precisamos descer.
    // Antes de descer, TEMOS que empurrar qualquer sujeira antiga
(push).
    push(node, start, end);

    int mid = (start + end) / 2;
    // Recurso para esquerda e direita
    update(2 * node, start, mid, l, r, val);
    update(2 * node + 1, mid + 1, end, l, r, val);

    // Na volta da recursao (backtracking), atualizamos o valor do no
atual
    // baseando-se nos valores atualizados dos filhos (Pull/Merge).
    tree[node] = tree[2 * node] + tree[2 * node + 1];
}

// Funcao publica para consulta
public long query(int l, int r) {
    return query(1, 0, n - 1, l, r);
}
    private long query(int node, int start, int end, int l, int r) {
        // Caso 1: Totalmente fora. Retorna elemento neutro (0 para soma).
        if (l > end || r < start) return 0;

        // Caso 2: Totalmente dentro. Retorna o valor armazenado.
        // Gracas ao 'push' e 'update' corretos, este valor ja considera
    todas as atualizacoes
        // relevantes ou tem um 'lazy' pendente que ja foi contabilizado
    no 'tree[node]'.
        if (l <= start && end <= r) return tree[node];

        // Caso 3: Parcial. Empurra pendencias antes de consultar os
    filhos.
        push(node, start, end);

        int mid = (start + end) / 2;
        // Soma os resultados das consultas nos filhos
        return query(2 * node, start, mid, l, r) +
                query(2 * node + 1, mid + 1, end, l, r);
    }
}
```

7.3 RangeMinimum

```
/*
    ================================================================================
 * Algoritmo: Segment Tree (Range Minimum Query - RMQ)
 * Complexidade: Construcao O(N). Update e Query O(log N). Memoria O(4*N).
 * Descricao: Encontra o valor minimo num intervalo [L, R] e atualiza
 * valores em indices especificos.
 * NOTA: O elemento neutro para queries de minimo e o INF (infinito).
 *
    ================================================================================
 */
#include <bits/stdc++.h>
using namespace std;

struct MinSegTree {
    int n;
    vector<long long> tree;
    const long long INF = 1e18;  // Elemento neutro para o minimo

    MinSegTree(int n) {
```

```
        this->n = n;
        tree.assign(4 * n, INF);
    }

    // Constri a  rvore  a partir de um array inicial em O(N)
    void build(const vector<long long>& a, int node, int left, int right)
    {
        if (left == right) {
            tree[node] = a[left];
            return;
        }
        int mid = left + (right - left) / 2;
        build(a, node * 2, left, mid);
        build(a, node * 2 + 1, mid + 1, right);
        tree[node] = min(tree[node * 2], tree[node * 2 + 1]);
    }

    // Atualiza um ndice especfico: arr[idx] = val
    void update(int node, int left, int right, int idx, long long val) {
        if (left == right) {
            tree[node] = val;
            return;
        }
        int mid = left + (right - left) / 2;
        if (idx <= mid) {
            update(node * 2, left, mid, idx, val);
        } else {
            update(node * 2 + 1, mid + 1, right, idx, val);
        }
        tree[node] = min(tree[node * 2], tree[node * 2 + 1]);
    }

    // Retorna o minimo no intervalo [ql, qr]
    long long query(int node, int left, int right, int ql, int qr) {
        // Intervalo totalmente fora da query
        if (left > qr || right < ql) return INF;

        // Intervalo totalmente contido na query
        if (left >= ql && right <= qr) return tree[node];

        // Intervalo parcialmente contido
        int mid = left + (right - left) / 2;
        long long res_left = query(node * 2, left, mid, ql, qr);
        long long res_right = query(node * 2 + 1, mid + 1, right, ql, qr);
        return min(res_left, res_right);
    }

    // Funcoes de interface limpas (0-indexed)
    void build(const vector<long long>& a) { build(a, 1, 0, n - 1); }
    void update(int idx, long long val) { update(1, 0, n - 1, idx, val); }
    long long query(int l, int r) { return query(1, 0, n - 1, l, r); }
};
```

8 SlidingWindow

8.1 Generic

```
/*
    ================================================================================
 * Problema Tipico: Maior Subarray com Restricao (Ex: Soma <= Target)
 * Algoritmo: Dois Ponteiros / Janela Variavel (Sliding Window)
 * Descricao: O ponteiro 'right' avanca sempre. O 'left' avanca apenas
    para
 * corrigir a janela quando esta se torna invalida.
 * Complexidade: O(N) Tempo (os ponteiros so andam para a frente)
 *
    ================================================================================
    */
#include <bits/stdc++.h>
using namespace std;

int generic_sliding_window(const vector<int>& arr, long long target) {
    int n = arr.size();
    int max_len = 0;
    long long current_sum = 0;
    int left = 0;

    for (int right = 0; right < n; right++) {
        // 1. Adiciona o elemento atual a janela
        current_sum += arr[right];

        // 2. Se a janela se tornou invalida, encolhe pela esquerda ate
    voltar a ser valida
        while (current_sum > target && left <= right) {
            current_sum -= arr[left];
            left++;
```

```
        }

        // 3. Neste ponto a janela [left, right] e garantidamente valida.
        // Atualiza a resposta (ex: tamanho maximo).
        max_len = max(max_len, right - left + 1);
    }

    return max_len;
}
```

8.2 Minimum

```
/*
    ============================================================================
 * Problema Tpico: Sliding Window Minimum / Maximum
 * Algoritmo: Monotonic Queue (Fila Monotnica) usando std::deque
 *  Descrio : Encontra o mnimo (ou mximo) em todas as janelas de
    tamanho K.
 * Complexidade: O(N) Tempo | O(K) Memria
 *
    ============================================================================
    */
#include <bits/stdc++.h>
using namespace std;

vector<int> sliding_window_minimum(const vector<int>& arr, int k) {
    vector<int> res;
    deque<int> dq; // Guarda os *ndices* dos elementos, no os valores

    for (int i = 0; i < arr.size(); i++) {
        // 1. Remove os ndices que j ficaram fora da janela atual [i - k
    + 1, i]
        if (!dq.empty() && dq.front() == i - k) {
            dq.pop_front();
        }

        // 2. Mantm a fila monotnica: remove pela direita tudo o que
    maior ou igual
        // ao elemento atual, pois nunca ser o mnimo desta janela nem das
    futuras.
        // (Para Sliding Window Maximum, basta mudar para arr[dq.back()]
    <= arr[i])
        while (!dq.empty() && arr[dq.back()] >= arr[i]) {
            dq.pop_back();
        }

        // 3. Insere o ndice do elemento atual
        dq.push_back(i);

        // 4. A partir do momento em que a primeira janela atinge o
    tamanho K,
        // o mnimo  estar  sempre na frente da deque.
        if (i >= k - 1) {
            res.push_back(arr[dq.front()]);
        }
    }
    return res;
}
```

9 Strings

9.1 PrefixFunction

```
/*
    ============================================================================
 * Algoritmo: KMP (Prefix Function / Pi Array)
 * Complexidade: O(N)
 *  Descrio : Encontra ocorrncias de um padro num texto em tempo linear.
 * Para usar em pattern matching, basta processar a string s =
    "padrao#texto".
 *
    ============================================================================
 */
#include <bits/stdc++.h>
using namespace std;

vector<int> compute_pi(const string& s) {
    int n = s.length();
    vector<int> pi(n, 0);
    for (int i = 1; i < n; i++) {
        int j = pi[i - 1];
        while (j > 0 && s[i] != s[j]) {
            j = pi[j - 1];  // Recua usando as bordas j calculadas
        }
        if (s[i] == s[j]) {
```

```
            j++;
        }
        pi[i] = j;
    }
    return pi;
}
```

9.2 PrefixTree

```
/*
    ================================================================================
 * Algoritmo: Trie (Prefix Tree) baseada em vetor esttico
 * Complexidade: O(L) por insero/pesquisa (L = tamanho da palavra)
 *  Descrio : Evita a sobrecarga de alocao  dinmica . Facilmente adaptvel
 * para uma Xor-Trie em problemas matemticos manipulando bits (0 e 1).
 *
    ================================================================================
 */
#include <bits/stdc++.h>
using namespace std;

struct TrieNode {
    int next[26];
    bool is_end;
    TrieNode() {
        fill(begin(next), end(next), -1);
        is_end = false;
    }
};

vector<TrieNode> trie(1);  // O n 0  sempre a raiz

void insert_string(const string& s) {
    int node = 0;
    for (char c : s) {
        int idx = c - 'a';  // Assumindo apenas minsculas
        if (trie[node].next[idx] == -1) {
            trie[node].next[idx] = trie.size();
            trie.emplace_back();  // Cria um novo n
        }
        node = trie[node].next[idx];
    }
    trie[node].is_end = true;
}

bool search_string(const string& s) {
    int node = 0;
    for (char c : s) {
        int idx = c - 'a';
        if (trie[node].next[idx] == -1) return false;
        node = trie[node].next[idx];
    }
    return trie[node].is_end;
}
```

9.3 RollingHashing

```
/*
    ================================================================================
 * Algoritmo: Rolling Hash com Duplo Hash (Rabin-Karp)
 * Complexidade: Pr- computao  O(N), Queries de Substring O(1)
 *  Descrio : Transforma substrings em inteiros para comparaes diretas.
 *
    ================================================================================
 */
#include <bits/stdc++.h>
using namespace std;

const long long M1 = 1e9 + 7, M2 = 1e9 + 9;
const long long B1 = 313, B2 = 317;

struct Hash {
    vector<long long> h1, h2, p1, p2;

    Hash(const string& s) {
        int n = s.length();
        h1.assign(n + 1, 0);
        h2.assign(n + 1, 0);
        p1.assign(n + 1, 1);
        p2.assign(n + 1, 1);

        for (int i = 0; i < n; i++) {
            p1[i + 1] = (p1[i] * B1) % M1;
            p2[i + 1] = (p2[i] * B2) % M2;

            h1[i + 1] = (h1[i] * B1 + s[i]) % M1;
```

```
            h2[i + 1] = (h2[i] * B2 + s[i]) % M2;
        }
    }

    // Retorna o par de hash da substring s[l...r] (0-indexed, inclusivo)
    pair<long long, long long> get(int l, int r) {
        long long hash1 = (h1[r + 1] - (h1[l] * p1[r - l + 1]) % M1 + M1)
    % M1;
        long long hash2 = (h2[r + 1] - (h2[l] * p2[r - l + 1]) % M2 + M2)
    % M2;
        return {hash1, hash2};
    }
};
```

9.4 Z-Algorithm

```
/*
    ================================================================================
 * Algoritmo: Z-Algorithm
 * Complexidade: O(N)
 *  Descrio : z[i]   o tamanho do maior prefixo comum entre s[0...N-1] e
 * s[i...N-1]. A  otimizao  baseia-se em manter um intervalo de
    correspondncia
 * [l, r].
 *
    ================================================================================
 */
#include <bits/stdc++.h>
using namespace std;

vector<int> z_algorithm(const string& s) {
    int n = s.length();
    vector<int> z(n, 0);
    int l = 0, r = 0;
    for (int i = 1; i < n; i++) {
        if (i <= r) {
            z[i] = min(r - i + 1, z[i - l]);
        }
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) {
            z[i]++;
        }
        if (i + z[i] - 1 > r) {
            l = i;
            r = i + z[i] - 1;
        }
    }
    return z;
}
```